

## Assignment No : 02

```
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

# 1. Load the dataset
df = pd.read_csv("Student.csv")

# Display first 5 records
print("First 5 Records:")
print(df.head())

# Display dataset information
print("\nDataset Information:")
print(df.info())

# 2. Select features and target
X = df[
    [
        "Age",
        "Class",
        "Study_Hours_Per_Day",
        "Attendance_Percentage",
        "Math_Score",
        "Science_Score",
        "English_Score",
        "Previous_Year_Score"
    ]
]

y = df["Pass_Fail"]

# 3. Split dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining Data Size:", X_train.shape)
print("Testing Data Size:", X_test.shape)

# 4. Create Logistic Regression model
model = LogisticRegression(max_iter=1000)

# 5. Train the model
model.fit(X_train, y_train)
```

```
# 6. Make predictions
y_pred = model.predict(X_test)

# 7. Evaluate the model
accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:")
print(accuracy)

print("\nAccuracy Percentage:")
print(accuracy * 100, "%")

# 8. Display confusion matrix
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# 9. Display classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns

# Create confusion matrix
cm = confusion_matrix(
    y_test,
    y_pred,
    labels=["Fail", "Pass"]
)

print("Confusion Matrix:")
print(cm)

# Plot confusion matrix
plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Fail", "Pass"],
    yticklabels=["Fail", "Pass"]
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()
```

## Output:

```
[1]: import pandas as pd
```

```
[2]: from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report
```

```
[3]: # 1. Load the dataset
df = pd.read_csv("Student.csv")
```

```
[4]: # Display first 5 records
print("First 5 Records:")
print(df.head())
```

First 5 Records:

|   | Student_ID | Age | Gender | Class | Study_Hours_Per_Day | Attendance_Percentage | \ |
|---|------------|-----|--------|-------|---------------------|-----------------------|---|
| 0 | S0001      | 15  | Male   | 12    | 1.0                 | 65                    |   |
| 1 | S0002      | 19  | Female | 9     | 1.6                 | 58                    |   |
| 2 | S0003      | 14  | Female | 12    | 3.6                 | 64                    |   |
| 3 | S0004      | 18  | Female | 9     | 5.5                 | 68                    |   |
| 4 | S0005      | 14  | Male   | 10    | 5.0                 | 80                    |   |

|   | Parental_Education | Internet_Access | Extracurricular_Activities | Math_Score | \  |
|---|--------------------|-----------------|----------------------------|------------|----|
| 0 | Postgraduate       |                 | No                         | No         | 40 |
| 1 | Graduate           |                 | No                         | Yes        | 80 |
| 2 | High School        |                 | Yes                        | Yes        | 83 |
| 3 | Postgraduate       |                 | Yes                        | No         | 68 |
| 4 | High School        |                 | Yes                        | No         | 41 |

|  | Science_Score | English_Score | Previous_Year_Score | Final_Percentage | \ |
|--|---------------|---------------|---------------------|------------------|---|
|--|---------------|---------------|---------------------|------------------|---|

```
[5]: # Display dataset information
print("\nDataset Information:")
print(df.info())
```

Dataset Information:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 5000 entries, 0 to 4999

Data columns (total 16 columns):

| #  | Column                     | Non-Null Count | Dtype   |
|----|----------------------------|----------------|---------|
| 0  | Student_ID                 | 5000 non-null  | object  |
| 1  | Age                        | 5000 non-null  | int64   |
| 2  | Gender                     | 5000 non-null  | object  |
| 3  | Class                      | 5000 non-null  | int64   |
| 4  | Study_Hours_Per_Day        | 5000 non-null  | float64 |
| 5  | Attendance_Percentage      | 5000 non-null  | int64   |
| 6  | Parental_Education         | 5000 non-null  | object  |
| 7  | Internet_Access            | 5000 non-null  | object  |
| 8  | Extracurricular_Activities | 5000 non-null  | object  |
| 9  | Math_Score                 | 5000 non-null  | int64   |
| 10 | Science_Score              | 5000 non-null  | int64   |
| 11 | English_Score              | 5000 non-null  | int64   |
| 12 | Previous_Year_Score        | 5000 non-null  | int64   |
| 13 | Final_Percentage           | 5000 non-null  | float64 |
| 14 | Performance_Level          | 5000 non-null  | object  |
| 15 | Pass_Fail                  | 5000 non-null  | object  |

dtypes: float64(2), int64(7), object(7)

memory usage: 625.1+ KB

None

```
[6]: # 2. Select features and target
```

```
X = df[
    [
        "Age",
        "Class",
        "Study_Hours_Per_Day",
        "Attendance_Percentage",
        "Math_Score",
        "Science_Score",
        "English_Score",
        "Previous_Year_Score"
    ]
]
```

```
[7]: y = df["Pass_Fail"]
```

```
[8]: # 3. Split dataset into training and testing sets
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

```
[9]: print("\nTraining Data Size:", X_train.shape)
```

```
print("Testing Data Size:", X_test.shape)
```

```
Training Data Size: (4000, 8)
```

```
Testing Data Size: (1000, 8)
```

```
[10]: # 4. Create Logistic Regression model
```

```
model = LogisticRegression(max_iter=1000)
```

```
[11]: # 5. Train the model
```

```
model.fit(X_train, y_train)
```

```
[11]: ▾ LogisticRegression ⓘ ?
```

```
▶ Parameters
```

```
[12]: # 6. Make predictions
```

```
y_pred = model.predict(X_test)
```

```
[13]: # 7. Evaluate the model
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
[14]: print("\nModel Accuracy:")
```

```
print(accuracy)
```

```
Model Accuracy:
```

```
1.0
```

```
[15]: print("\nAccuracy Percentage:")
```

```
print(accuracy * 100, "%")
```

```
Accuracy Percentage:
```

```
100.0 %
```

```
[16]: # 8. Display confusion matrix
```

```
print("\nConfusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
```

Confusion Matrix:

```
[[ 53  0]
 [ 0 947]]
```

```
[17]: # 9. Display classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Classification Report:

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| Fail         | 1.00      | 1.00   | 1.00     | 53      |
| Pass         | 1.00      | 1.00   | 1.00     | 947     |
| accuracy     |           |        | 1.00     | 1000    |
| macro avg    | 1.00      | 1.00   | 1.00     | 1000    |
| weighted avg | 1.00      | 1.00   | 1.00     | 1000    |

```
[18]: pip install seaborn
```

```
Requirement already satisfied: seaborn in d:\python\lib\site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in d:\python\lib\site-packages (from seaborn) (2.2.6)
Requirement already satisfied: pandas>=1.2 in d:\python\lib\site-packages (from seaborn) (2.3.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in d:\python\lib\site-packages (from seaborn) (3.10.8)
Requirement already satisfied: contourpy>=1.0.1 in d:\python\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.2)
Requirement already satisfied: cycler>=0.10 in d:\python\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in d:\python\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in d:\python\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.5.0)
Requirement already satisfied: packaging>=20.0 in d:\python\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.0)
Requirement already satisfied: pillow>=8 in d:\python\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.1.1)
Requirement already satisfied: pyparsing>=3 in d:\python\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in d:\python\lib\site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
```

```
[19]: import seaborn as sns
import matplotlib.pyplot as plt
```

```
[20]: cm = confusion_matrix(y_test, y_pred)
```

```
[21]: # Plot confusion matrix
plt.figure(figsize=(6, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Fail", "Pass"],
    yticklabels=["Fail", "Pass"]
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.tight_layout()
plt.show()
```



